

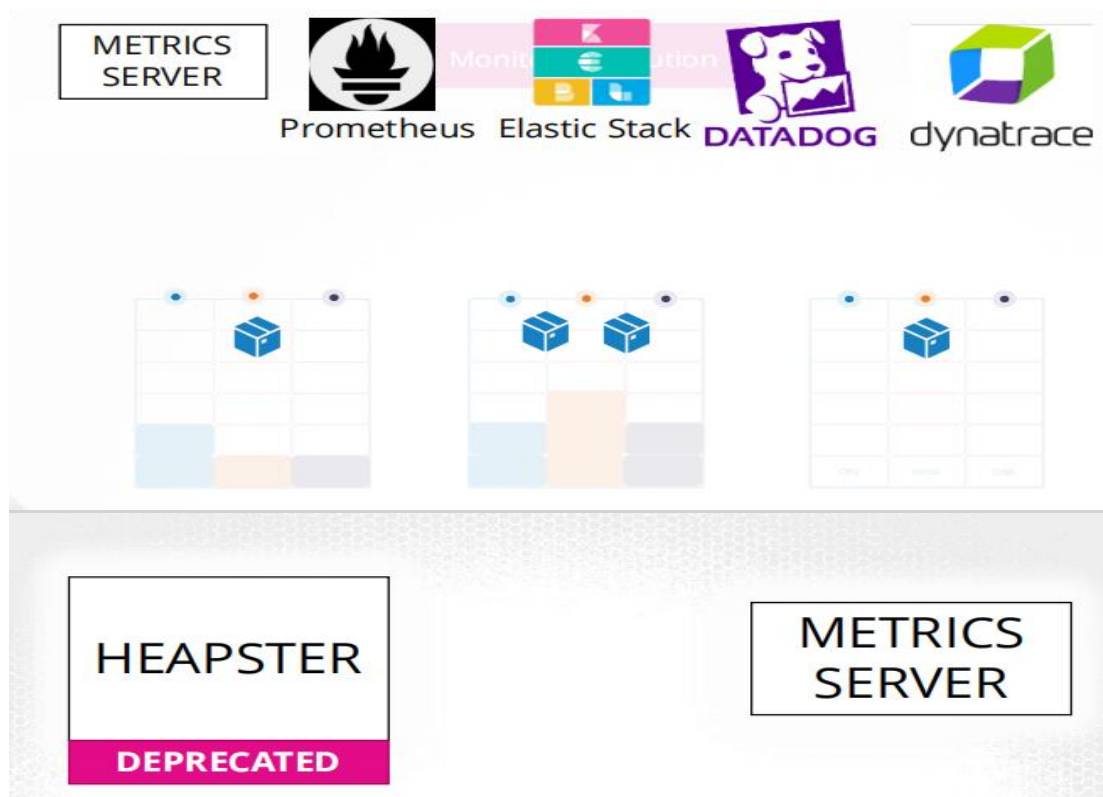
Monitor

How do you monitor resource consumption on Kubernetes? Or more importantly what **would you like to monitor**? I'd like to know Node level metrics such as the number of nodes in the cluster, how many of them are healthy as well as performance metrics such as CPU, memory, and network and disk utilization. As well as POD level metrics such as the number of PODs, and performance metrics of each POD such the CPU and Memory consumption.

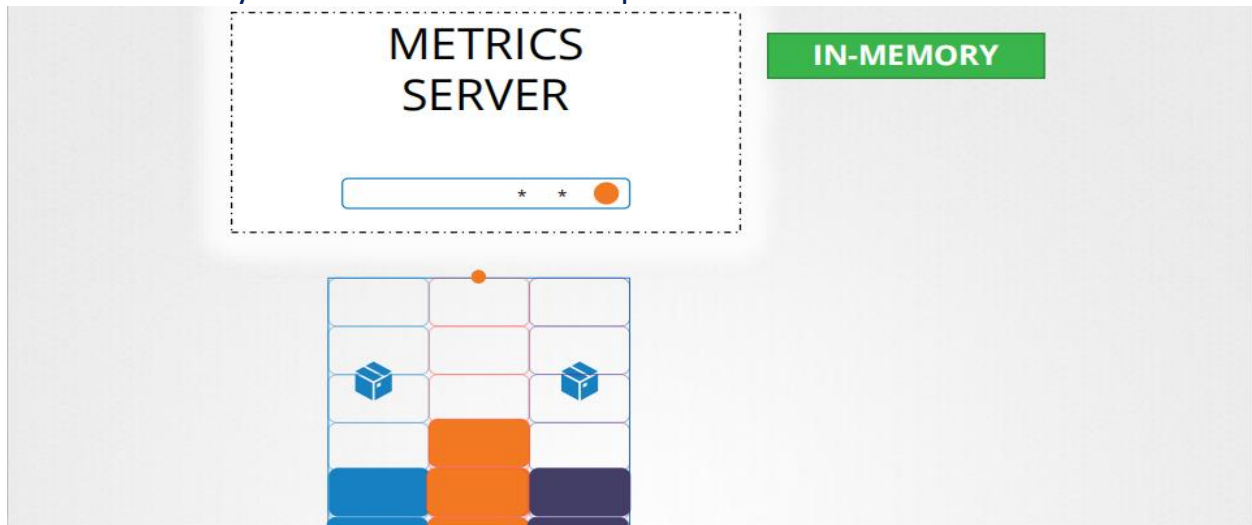


So, we need a solution that will monitor these metrics, store them and provide analytics around this data.

As of now k8s does not comes with such type of solution, while there are lots of third party, open-source solutions are available today, such as the **Metrics-Server**, Prometheus, the Elastic Stack, and proprietary solutions like Datadog and Dynatrace. You have one metrics server per K8s cluster.



The metrics server retrieves metrics from each of the kubernetes nodes and pods, aggregates them and stores them in memory. Note that the metrics server is only an **in-memory monitoring solution** and does not store the metrics on the disk, and as a result you cannot see historical performance data.



Solution: you have to rely on one of the advance monitoring solutions.

How are the metrics generated for the PODs on these nodes?

Kubernetes runs an agent on each node known as the kubelet, which is responsible for receiving instructions from the kubernetes API master server and running PODs on the nodes. **The kubelet also contains a subcomponent known as cAdvisor or Container Advisor, cAdvisor is responsible for retrieving performance metrics from pods, and exposing them through the kubelet API to make the metrics available for the Metrics Server.**

METRICS SERVER



others

```
git clone https://github.com/kubernetes-incubator/metrics-server.git
```

```
kubectl create -f deploy/1.8+/
```

```
clusterrolebinding "metrics-server:system:auth-delegator" created
rolebinding "metrics-server-auth-reader" created
apiservice "v1beta1.metrics.k8s.io" created
serviceaccount "metrics-server" created
deployment "metrics-server" created
service "metrics-server" created
clusterrole "system:metrics-server" created
clusterrolebinding "system:metrics-server" created
```

```
kubectl top node
```

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
kubemaster	166m	8%	1337Mi	70%
kubenode1	36m	1%	1046Mi	55%
kubenode2	39m	1%	1048Mi	55%

```
kubectl top pod
```

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
nginx	166m	8%	1337Mi	70%
redis	36m	1%	1046Mi	55%

General monitoring concepts: I would like to monitor my production infrastructure; we have to monitor different components separately so that we can identify the root cause of the issue.

- System level monitoring
 - cpu/ram/disk/ping/router/network io
- Process level monitoring
 - docker/tomcat/kubelet/cron/db/ldap/email/dns/kafka/
 - cpu/ram/pods
- Application-level monitoring
 - availability/ response time/ transaction time of application

Development

— git/github

Operations

— build code - Jenkins/maven
— config mgmt - ansible
— iac - terraform
— cont — docker
— cont orch — kubernetes

— db admin
— network admin
— storage admin

Maintenance

logging tools
splunk/elk
monitoring tools
prometheus / grafana

nagios
ca apm
dynatrace
app dynamics
sysdig
new relic
sitescope / bsm
appd / zabbix

monitoring

— identifying the prob in your infra (prod env)

what to monitor ?

System Level

— ping/router/switch/disk/cpu/mem/IO/users

process / service Level

— web/appservers/docker/kubelet/db/LDAP/
storage/LB/Firewall

app Level

— url / response time / transaction time / logs

Prometheus help us in System level and Process level monitoring.

Prometheus: monitoring tool (system and process level), with Prometheus, helps us at k8s cluster level whether my pods are available or not, how much CPU taken by my pods.

Prometheus is a monitoring tool it has to install somewhere, it will collect metrics of the current status of pods running in your k8s cluster using it, it has to some input, it has to go somewhere to fetch the information, by default api server has all information available but by default api server don't have capability to provide the metrics, api server have already lots of other things to do, so prometheus has not to go to api server now instead of that we have to install

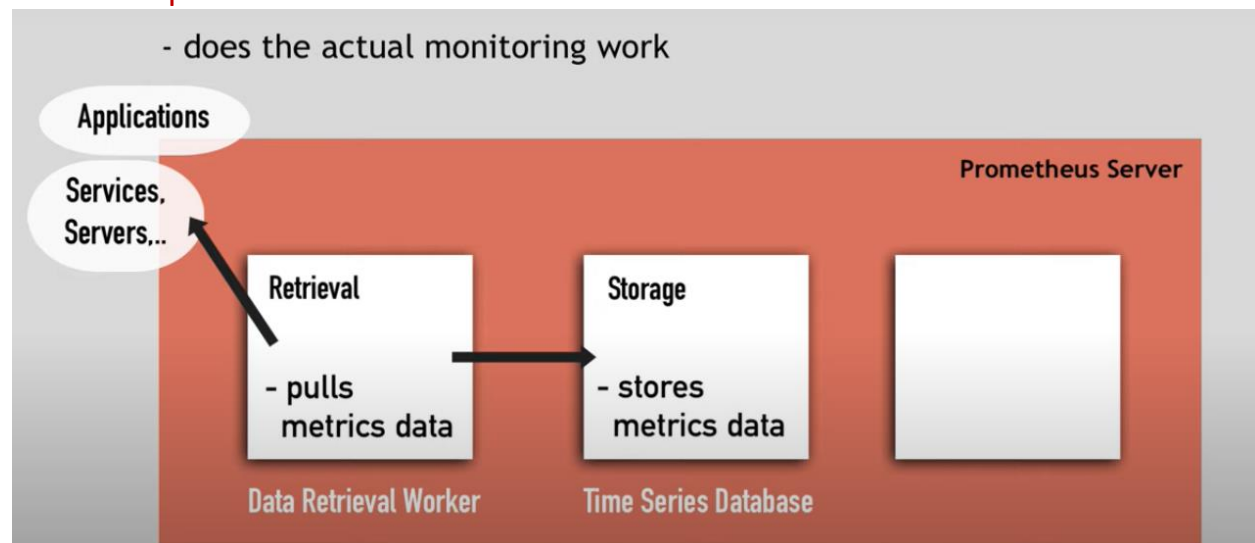
another component purely collects the metrics from api server , which talks to api server and continuous collects the all metrics, that is called **kube-state-metrics** (add on)which is given by k8s again ,which collects the metrics from api server is some interval.

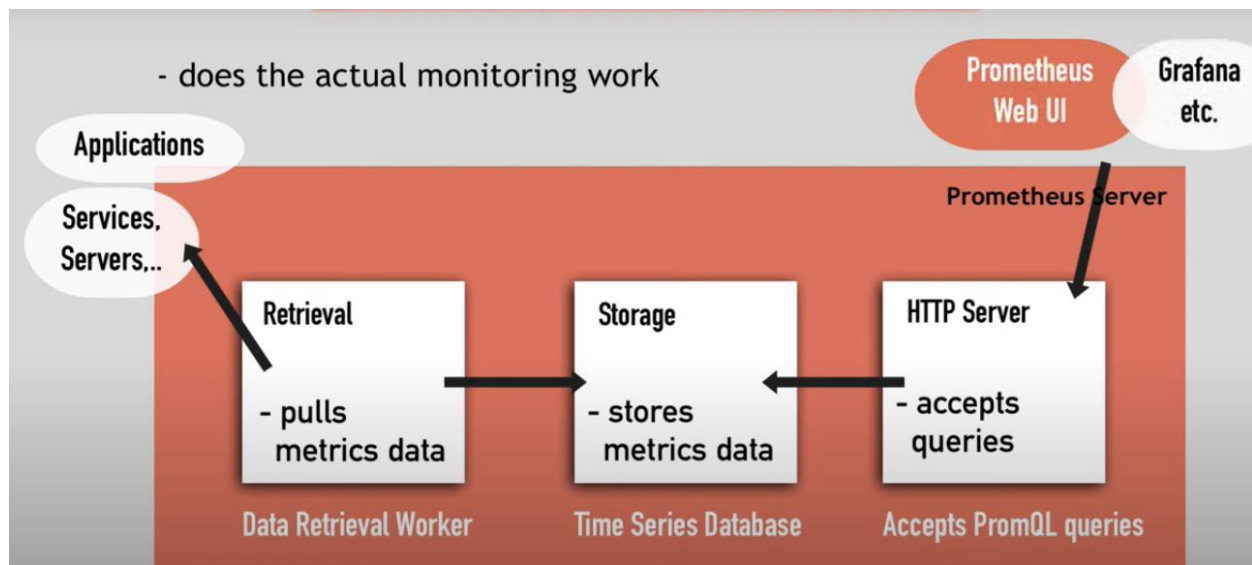
Once we deploy only then it's going to collect all the metrics from the cluster. But this don't have db to store the metrics which it is collecting.

Now **Prometheus** comes in picture, Prometheus talk with **kube-state metrics** and keeps on collecting, it works on pull model, it pulls the metrics from kube-state-metrics and store the metrics in its in-build default database **influxDB**. Everything is stored in its db influxDB continuously, it stores all the metrics now it has history of metrics.

Prometheus can't directly collect the metrics, k8s created a metrics server **kube-state metrics**

Main Components: Prometheus Server





```

root@master:~/educka/monitoring/prometheus# kubectl -n monitoring get all
NAME                                READY   STATUS    RESTARTS   AGE
pod/grafana-7b6c9c59c8-vqpgr       1/1     Running   0          14h
pod/prometheus-deployment-75d9648b57-wlwdg  1/1     Running   0          22s

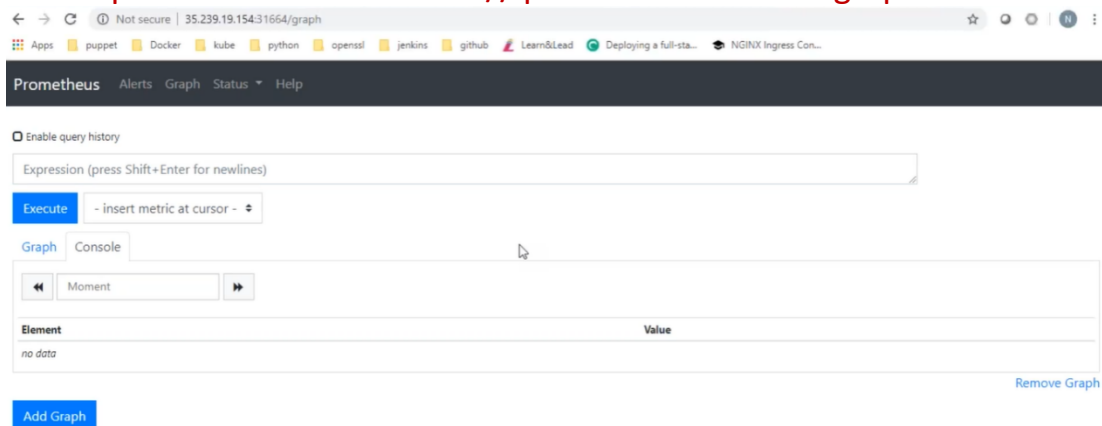
NAME                                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/grafana-ui-svc              NodePort      10.111.190.36   <none>            3000:31872/TCP   14h
service/prometheus-service          NodePort      10.100.36.148   <none>            8080:31664/TCP   22s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/grafana              1/1     1             1           14h
deployment.apps/prometheus-deployment 1/1     1             1           22s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/grafana-7b6c9c59c8   1         1         1       14h
replicaset.apps/prometheus-deployment-75d9648b57 1         1         1       22s
  
```

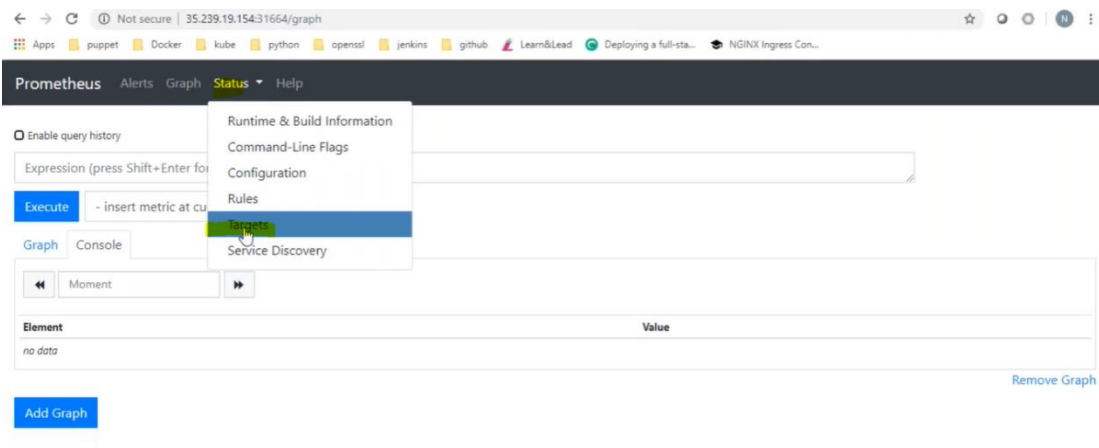
prometheus – is best monitoring tool, good collector but don't have rich UI

grafana- have rich UI // k8s dashboard for grafana 6417 /11141 (for pod)
 <nodeip: NodePort> // prometheus default graph dashboard



status->target

// target from it collects the metrics



Prometheus doesn't have rich dashboard, it shows only default metrics, it has all the data but don't have rich dashboard.

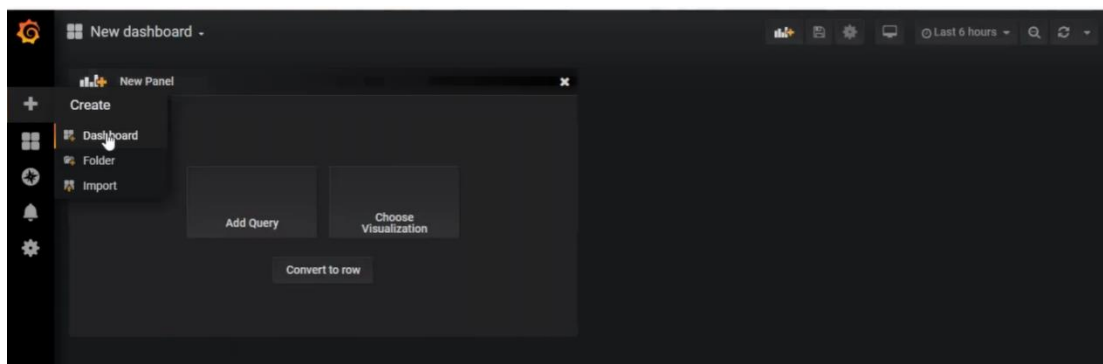
How to show rich dashboard?

Grafana Is all about how to visualize the data better, it collects the data from Prometheus db, need to connect with the Prometheus (as a data source), it will pull the data from Prometheus db.

login: admin/admin, add data source: new dashboard:

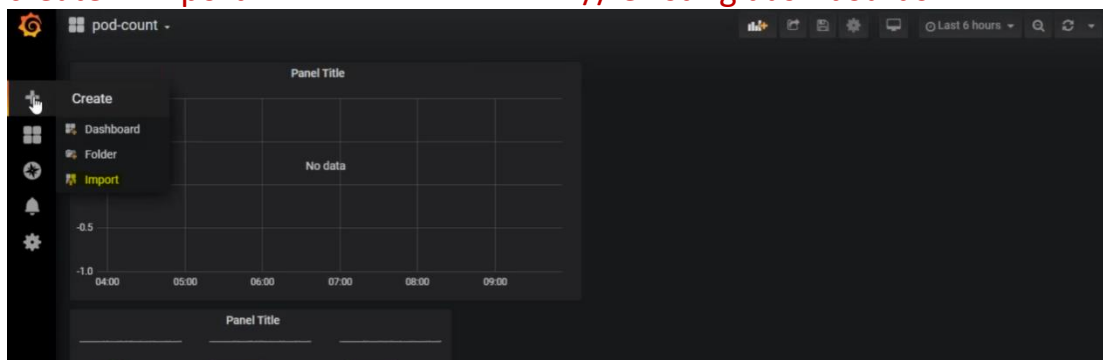
How to configure Grafana dash board?

Create-> new dash board



create-> import

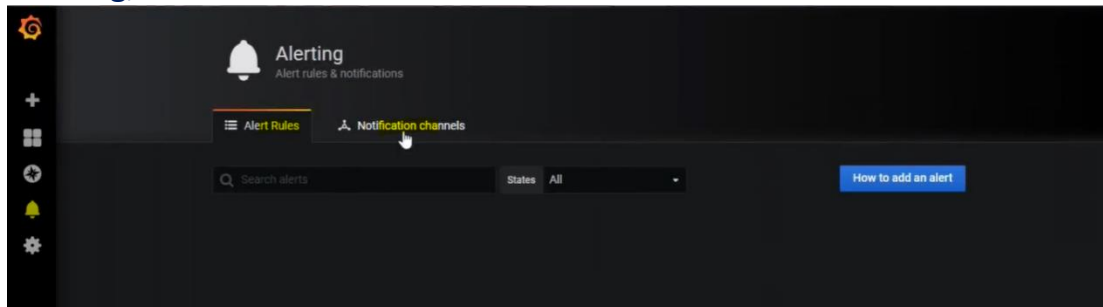
// existing dash boards



6417, 11141

How can we configure mail server?

Inside your grafana pod, first grafana mailing service to be enable.
Alerting, Notification channels



Managing Applications Logs?

Logs-Docker

`docker run <image>`

UC: If we run the container in detached mode, we can't view the log.

`docker run -d <image> // kodekloud/event-simulator`

`docker logs <container id>`

`docker logs -f <container id> // to see the live logs`

Logs-K8s

`Kubectl logs -f < name of the pod> -c <name of the container>`

`kubectl logs etcd-master -n kube-system`

If pod is not available go one step behind

```
▶ docker ps -a
```

CONTAINER ID	STATUS	NAMES
23482a09f25b	Up 12 minutes	k8s_kube-apiserver_kube-apiserver-master_kube-system_8758a3d10776bb527
b9bf77348c96	Up 18 minutes	k8s_etcd_etcd-master_kube-system_2cc1c8a24b68ab9b46bca47e153e74c6_0
87fc69913973	Up 18 minutes	k8s_POD_etcd-master_kube-system_2cc1c8a24b68ab9b46bca47e153e74c6_0
fda322157b86	Exited (255) 18 minutes ago	k8s_kube-apiserver_kube-apiserver-master_kube-system_8758a3d10776bb527
0794bdfd57d8	Up 40 minutes	k8s_kube-scheduler_kube-scheduler-master_kube-system_009228e74aef4d7ba
00f3f95d2102	Up 40 minutes	k8s_kube-controller-manager_kube-controller-manager-master_kube-system
b8e6a0e173dd	Up About an hour	k8s_weave_weave-net-8dzw kube-system_22cd7993-2f2d-11e9-a2a6-0242ac1f
18e47bad320e	Up About an hour	k8s_weave-npc_weave-net-8dzw kube-system_22cd7993-2f2d-11e9-a2a6-0242
4d087daf0380	Exited (1) About an hour ago	k8s_weave_weave-net-8dzw kube-system_22cd7993-2f2d-11e9-a2a6-0242ac1f

▶ docker logs 87fc

```
2019-02-13 02:53:28.144631 I | etcdmain: etcd Version: 3.2.18
2019-02-13 02:53:28.144680 I | etcdmain: Git SHA: eddf599c6
2019-02-13 02:53:28.144684 I | etcdmain: Go Version: go1.8.7
2019-02-13 02:53:28.144688 I | etcdmain: Go OS/Arch: linux/amd64
2019-02-13 02:53:28.144692 I | etcdmain: setting maximum number of CPUs to 4, total number of available CPUs is 4
2019-02-13 02:53:28.144734 N | etcdmain: the server is already initialized as member before, starting as etcd
member...
2019-02-13 02:53:28.146625 I | etcdserver: name = master
2019-02-13 02:53:28.146637 I | etcdserver: data dir = /var/lib/etcd
2019-02-13 02:53:28.146642 I | etcdserver: member dir = /var/lib/etcd/member
2019-02-13 02:53:28.146645 I | etcdserver: heartbeat = 100ms
2019-02-13 02:53:28.146648 I | etcdserver: election = 1000ms
2019-02-13 02:53:28.146651 I | etcdserver: snapshot count = 10000
2019-02-13 02:53:28.146677 I | etcdserver: advertise client URLs = 2019-02-13 02:53:28.185353 I | etcdserver/api:
enabled capabilities for version 3.2
2019-02-13 02:53:28.185588 I | embed: ClientTLS: cert = /etc/kubernetes/pki/etcd/server.crt, key =
/etc/kubernetes/pki/etcd/server.key, ca = , trusted-ca = /etc/kubernetes/pki/etcd/old-ca.crt, client-cert-auth =
```